

Analisi morfologica e classificazione non-CNN di lettere manoscritte

Studente: _____

27 aprile 2026

Sommario

Questo lavoro propone una pipeline completa e interpretabile per il riconoscimento di lettere manoscritte basata su Morfologia Matematica. L'idea centrale è separare in modo esplicito le fasi di: (i) pulizia dell'immagine, (ii) estrazione di struttura geometrica/topologica, (iii) classificazione supervisionata. Le immagini sono binarizzate (Otsu), scheletrizzate e potate (pruning) per ridurre il rumore mantenendo l'informazione di forma. Da queste rappresentazioni vengono estratte feature robuste (topologiche, geometriche, spaziali, statistiche), poi usate da modelli non-CNN (SVM RBF, ExtraTrees, XGBoost). La valutazione è svolta con split per soggetto e con cross-validation subject-wise, per misurare la reale capacità di generalizzare a calligrafie mai viste. Il miglior risultato holdout è ottenuto da XGBoost (Accuracy 0.7369, Macro-F1 0.7338), confermando che il feature engineering morfologico è competitivo e ben motivato.

1 Introduzione

Il riconoscimento di scrittura manoscritta è un problema complesso per tre motivi principali:

- **variabilità inter-soggetto:** la stessa lettera può avere forme molto diverse;
- **rumore di acquisizione:** sfondo, spessori irregolari, piccole sbavature;
- **ambiguità intrinseca:** alcune coppie di lettere sono naturalmente simili.

Invece di una soluzione end-to-end con rete neurale profonda, questo progetto adotta un approccio **esplicativo**: ogni passaggio è leggibile e ogni feature ha un significato geometrico o topologico chiaro.

Obiettivi.

- Ridurre il rumore preservando la struttura del tratto.
- Estrarre descrittori topologici e geometrici robusti.
- Validare la generalizzazione tra soggetti diversi con protocollo subject-wise.
- Mantenere una pipeline riproducibile e interpretabile in tutte le fasi.

2 Dataset e organizzazione

Il dataset è organizzato per soggetto e classe, nella forma:

```
data/lettere/<soggetto>/<lettera>/<immagine>
```

Questa struttura permette di tenere separata l'identità del soggetto dalla classe della lettera, condizione fondamentale per evitare data leakage quando si fa split train/test.

Statistiche principali:

- **14521** campioni totali,
- **25** classi di lettere,
- **20** soggetti.

In pratica, ogni campione attraversa la stessa pipeline morfologica e viene trasformato in un vettore numerico uniforme, poi aggregabile per analisi globale o per soggetto.

2.1 Preparazione del dataset: estrazione caratteri da CVL

La prima fase del progetto non è la classificazione, ma la costruzione del dataset a partire dalle pagine manoscritte. Per questo è stato aggiunto uno script dedicato, `extract_characters_cv1.py` che trasforma una pagina completa in singoli crop di carattere pronti per l'analisi successiva.

In altre parole, questa fase fa da ponte tra i manoscritti grezzi e la pipeline morfologica: senza segmentazione dei caratteri, non avremmo campioni omogenei su cui calcolare le feature e addestrare i modelli.

Lo script opera su immagine singola o in batch e realizza:

1. conversione in grayscale,
2. binarizzazione adattiva con soglia gaussiana e inversione del foreground,
3. denoising per attenuare piccoli artefatti e rumore impulsivo,
4. erosione e dilatazione per ripulire il tratto e stabilizzare i contorni,
5. eventuale opening/closing per rimuovere micro-componenti isolate o chiudere piccole interruzioni,
6. estrazione delle componenti connesse e delle bounding box candidate,
7. filtraggio geometrico su area, larghezza, altezza e aspect ratio,
8. filtro outlier basato su mediane (area/altezza) per scartare box troppo grandi rispetto al comportamento tipico della pagina,
9. ordinamento in ordine di lettura (riga per riga, da sinistra a destra),
10. salvataggio dei crop carattere per carattere,
11. esportazione del file `extraction_manifest.csv` con coordinate (x, y, w, h).

Dal punto di vista operativo, quindi, l'immagine viene prima normalizzata e ripulita, poi segmentata in regioni candidate, e infine rifinita con controlli geometrici per ottenere crop di carattere più consistenti e utili alle fasi successive.

Questa fase è coerente con il progetto perché rende esplicito il passaggio da pagina manoscritta a campioni elementari (caratteri) su cui poi si applica la pipeline morfologica e la classificazione.

Configurazione consigliata (quality profile). Per ottenere una segmentazione più pulita sui manoscritti CVL, la configurazione consigliata è:

- `-min-area 30`
- `-min-width 5`
- `-min-height 8`
- `-max-aspect-ratio 10`
- `-opening-kernel 2`

Questa scelta riduce in modo sensibile micro-componenti spurie e rettangoli anomali, mantenendo i caratteri utili per le fasi successive di feature extraction e classificazione.

Dal punto di vista del flusso complessivo, questa è la parte che precede tutto il resto: prima estraggo i caratteri, poi applico la morfologia, poi costruisco le feature e infine alleno i modelli.

3 Metodologia

3.1 Preprocessing e morfologia

La pipeline operativa include quattro passaggi principali:

1. **Conversione e binarizzazione (Otsu).** L'immagine in scala di grigi viene separata in foreground/sfondo con una soglia automatica. Questo rende robusta la segmentazione del tratto senza fissare soglie manuali.
2. **Skeletonization.** Il tratto binario viene ridotto a uno scheletro a 1 pixel di spessore, preservando connettività e struttura globale.
3. **Pruning iterativo.** Si rimuovono iterativamente endpoint troppo corti (rami spuri), in modo da attenuare rumore locale e micro-ramificazioni non informative.
4. **Estrazione feature.** Le misure sono calcolate sia sull'immagine binaria sia sullo scheletro ripulito, così da catturare informazioni complementari.

3.2 Perché la morfologia è centrale

Le operazioni morfologiche non sono un semplice preprocessing “estetico”: costituiscono la base semantica del modello. In particolare:

- la binarizzazione definisce il dominio su cui misurare area, densità e cavità;

- lo scheletro rende esplicita la topologia del carattere (terminazioni, incroci, lunghezza dei rami);
- il pruning aumenta il rapporto segnale/rumore delle feature strutturali.

Senza questi passaggi, molte feature perderebbero stabilità e interpretabilità.

3.3 Feature estratte

Le feature sono organizzate in gruppi complementari. Di seguito sono descritte con il loro significato operativo.

1) Feature base/topologiche

- **endpoints / punte**: numero di terminazioni dello scheletro; utile per distinguere lettere aperte/chiose o con code.
- **junctions / incroci**: numero di nodi con più rami; discriminante per lettere con intersezioni interne.
- **holes / buchi**: numero di cavità chiuse nel tratto (es. “o”, “p”, “b”).
- **componenti**: numero di componenti connesse, utile in presenza di tratti separati.
- **euler_number**: sintetizza topologia globale (componenti meno buchi).

2) Feature geometriche di base

- **aspect_ratio**: rapporto larghezza/altezza della bounding box della componente principale.
- **densita_pixel**: frazione di pixel attivi sull’area totale.
- **avg_stroke_width**: stima dello spessore medio del tratto (distance transform + scheletro).
- **skeleton_length**: lunghezza complessiva del tratto scheletrizzato.
- **skeleton_density**: densità dello scheletro rispetto all’ink totale.
- **endpoint_norm, junction_norm**: endpoint/incroci normalizzati per lunghezza scheletro.
- **pruning_removed_ratio**: quota di scheletro rimossa dal pruning (indicatore di rumore locale).

3) Feature di forma globale e invarianti

- **solidity**: rapporto area/convex hull area, misura “pienezza” della forma.
- **extent**: rapporto area/area bounding box.
- **eccentricity**: allungamento della forma principale.
- **orientation**: orientazione prevalente dell’asse maggiore.
- **major_axis_length, minor_axis_length, axis_ratio**: parametri ellittici della componente principale.
- **hu_1 ... hu_7**: momenti invarianti di Hu (log-compressi), robusti a traslazione/scala/rotazione.

4) Feature di struttura del grafo scheletrico

- **n_branches**: numero di rami ottenuti rimuovendo i nodi di giunzione.
- **branch_length_mean, branch_length_max, branch_length_std**: statistiche di lunghezza dei rami.

5) Feature spaziali

- **zoning 3x3 (zone_r*_c*_density)**: densità locale in ciascuna cella della griglia.
- **projection profiles**: statistiche dei profili orizzontali e verticali.
- **spatial balance**: bilanciamento top/bottom e left/right (es. **top_bottom_ratio, left_right_ratio**).

6) Feature strutturali avanzate

- **transition features** (riga/colonna): numero e statistiche di transizioni 01, indicano complessità del contorno.
- **diagonal profile**: densità su diagonale principale e secondaria, con differenza tra le due.
- **endpoint position**: posizione media/dispersione normalizzata delle terminazioni.

7) Feature confusion-driven Queste feature sono pensate per migliorare coppie notoriamente ambigue:

- **scanline structure**: pattern locali su righe campione;
- **border contact**: come il tratto tocca i bordi dell’immagine normalizzata;
- **hole position**: posizione dei buchi (non solo quantità);
- **contour depth**: profondità media del tratto verso l’interno;
- **endpoint distribution**: distribuzione spaziale delle terminazioni;
- **midline runs**: lunghezza dei segmenti continui su linee mediane.

8) Statistiche aggiuntive sulle proiezioni Per `row_proj` e `col_proj` vengono aggiunti:

- quantili (q_{25}, q_{50}, q_{75}),
- asimmetria (skewness),
- curtosi (kurtosis).

Queste misure migliorano la descrizione della distribuzione del tratto oltre la sola media/varianza.

3.4 Spiegazione esplicita del *perché* di ogni feature

In questa sezione ogni famiglia è collegata direttamente al tipo di errore che aiuta a ridurre.

Topologia (`punte`, `incroci`, `buchi`, `componenti`, `euler_number`).

- `punte`: distingue lettere con code/terminazioni nette (es. tratti finali aperti) da forme più chiuse.
- `incroci`: separa lettere con struttura ramificata da lettere “a tratto unico”.
- `buchi`: discrimina lettere con anello interno da lettere senza cavità.
- `componenti`: identifica casi con tratti disconnessi (utile in presenza di scrittura spezzata o rumore).
- `euler_number`: sintetizza in un solo valore la relazione `componenti/buchi`, rendendo più robusta la distinzione topologica.

Geometria globale (`aspect_ratio`, `densita_pixel`, `avg_stroke_width`, `skeleton_length`, `skeleton_density`, `endpoint_norm`, `junction_norm`, `pruning_removed_ratio`).

- `aspect_ratio`: separa lettere tendenzialmente “alte” da lettere “larghe”.
- `densita_pixel`: misura quanta “inchiostrazione” c’è; utile per differenze di riempimento del gesto.
- `avg_stroke_width`: cattura lo stile di pressione/spessore del tratto.
- `skeleton_length`: proxy della complessità/estensione del percorso scritto.
- `skeleton_density`: confronta lunghezza scheletro e area inchiostrata, distinguendo tratti sottili da forme compatte.
- `endpoint_norm`, `junction_norm`: rendono confrontabili lettere di dimensione diversa grazie alla normalizzazione.
- `pruning_removed_ratio`: indica quanto rumore ramificato era presente; è informativa per stili tremolanti o sbavati.

Forma principale (**solidity**, **extent**, **eccentricity**, **orientation**, **major_axis_length**, **minor_axis_length**, **axis_ratio**).

- **solidity**: distingue forme compatte da forme con rientranze.
- **extent**: misura quanto la lettera occupa la propria bounding box.
- **eccentricity** e **axis_ratio**: misurano l'allungamento e aiutano su coppie con simile topologia ma proporzioni diverse.
- **orientation**: cattura inclinazione prevalente (utile su calligrafie inclinate).
- **major/minor_axis_length**: aggiungono scala e forma ellittica della componente dominante.

Invarianti di momento (**hu_1 ... hu_7**). Le Hu moments sono utili perché conservano informazione di forma anche in presenza di piccole variazioni geometriche; riducono la sensibilità a traslazione/scala/rotazione e quindi migliorano la robustezza inter-soggetto.

Struttura dei rami (**n_branches**, **branch_length_mean**, **branch_length_max**, **branch_length_std**). Queste feature descrivono la “grammatica” interna dello scheletro: quante diramazioni ci sono e quanto sono lunghe/variabili. Sono particolarmente utili quando due lettere hanno area simile ma struttura interna differente.

Distribuzione spaziale locale (**zone_r*_c*_density**, **row_proj_***, **col_proj_***, **top_bottom_ratio**, **left_right_ratio**).

- lo **zoning 3x3** localizza dove si concentra il tratto;
- i **projection profiles** descrivono il profilo verticale/orizzontale della forma;
- il **spatial balance** separa lettere “sbilanciate in alto/basso” o “a sinistra/destra”.

Insieme riducono confusioni tra lettere con stessa area totale ma diversa disposizione del tratto.

Complessità del contorno (**row_trans_***, **col_trans_***, **main_diag_density**, **anti_diag_density**, **diag_density_diff**). Le transizioni 01 quantificano quante volte il segnale entra/esce dal tratto lungo righe/colonne: più transizioni implicano contorni più complessi. Le diagonali aggiungono sensibilità alla direzione dominante del gesto, utile per lettere inclinate o oblique.

Posizionamento delle terminazioni (**endpoint_x_mean_norm**, **endpoint_y_mean_norm**, **endpoint_x_std_norm**, **endpoint_y_std_norm**, **endpoint_distribution_***). Queste misure spiegano *dove* finiscono i tratti: terminazioni concentrate in alto, in basso o distribuite su tutta la lettera. Sono utili per separare lettere con stesso numero di endpoint ma disposizione differente.

Feature confusion-driven (`scanline_*`, `border_contact_*`, `hole_position_*`, `contour_depth_*`, `midline_run_*`). Sono state introdotte per migliorare coppie ambigue osservate in confusion matrix.

- `scanline_*`: confrontano pattern locali su linee specifiche.
- `border_contact_*`: rilevano se il tratto tocca certi bordi (informazione discriminativa di forma aperta/chiusa).
- `hole_position_*`: aggiungono la posizione delle cavità, non solo il conteggio.
- `contour_depth_*`: misurano “profondità” e penetrazione del tratto rispetto al bordo.
- `midline_run_*`: quantificano segmenti continui sulle linee mediane, utili per distinguere andamenti centrali del tratto.

Statistiche di forma dei profili (`row/col_proj_q25/q50/q75`, `row/col_proj_skew`, `row/col_proj_kurtosis`). Queste statistiche non descrivono solo “quanto” tratto è presente, ma *come* è distribuito: asimmetria e curtosi aiutano a distinguere profili con stessa media ma forma diversa.

4 Setup sperimentale

4.1 Split holdout per soggetto

Per testare la generalizzazione reale, la divisione è fatta sui soggetti e non sui singoli campioni. Questo significa che il modello in test vede stili grafici completamente nuovi.

- **train**: 15 soggetti (11610 campioni)
- **test**: 5 soggetti (2911 campioni)

4.2 Cross-validation subject-wise

Per robustezza statistica è usata anche una CV a 5 fold subject-wise. Ogni fold mantiene la separazione per soggetto, così la varianza tra fold riflette differenze di stile realistiche.

4.3 Modelli e metriche

Sono confrontati tre modelli con ipotesi diverse:

- **SVM RBF**: adatto a separazioni non lineari in spazi ad alta dimensione;
- **ExtraTrees**: ensemble ad albero molto robusto a feature eterogenee;
- **XGBoost**: boosting gradient-based, spesso molto efficace su feature engineering tabellare.

Metriche principali:

- **Accuracy**: quota complessiva di predizioni corrette;
- **Macro-F1**: media F1 per classe con peso uniforme (fondamentale in presenza di difficoltà diverse tra lettere).

Completano l’analisi: report per classe, confusion matrix, F1 per lettera.

5 Risultati

5.1 Risultati holdout aggiornati

Tabella 1: Confronto modelli su holdout subject-wise

Modello	Accuracy	Macro-F1
XGBoost	0.7369	0.7338
SVM RBF	0.7248	0.7239
ExtraTrees	0.7238	0.7205

Il miglior modello in holdout risulta **XGBoost**. La differenza, pur non enorme, è coerente su entrambe le metriche principali e indica una migliore capacità di sfruttare la varietà delle feature estratte.

5.2 Risultati CV subject-wise (riferimento)

Tabella 2: CV 5-fold subject-wise (media $\pm$ deviazione standard)

Modello	Accuracy	Macro-F1
XGBoost	0.7676 ± 0.0246	0.7236 ± 0.0269
SVM RBF	0.7372 ± 0.0427	0.7039 ± 0.0351
ExtraTrees	0.7472 ± 0.0265	0.6898 ± 0.0306

In CV si osserva una buona stabilità del modello migliore: la deviazione standard resta contenuta e conferma che il comportamento non dipende da un singolo split favorevole.

5.3 Ablation study dei gruppi di feature

Per quantificare il contributo dei diversi blocchi di feature è stato eseguito un ablation study con XGBoost: si rimuove un gruppo alla volta e si misura la variazione di performance rispetto alla configurazione `all_features`.

Nel setup di riferimento dell'ablation (stesso split subject-wise in cache) la baseline è:

- Accuracy = 0.7269
- Macro-F1 = 0.7250

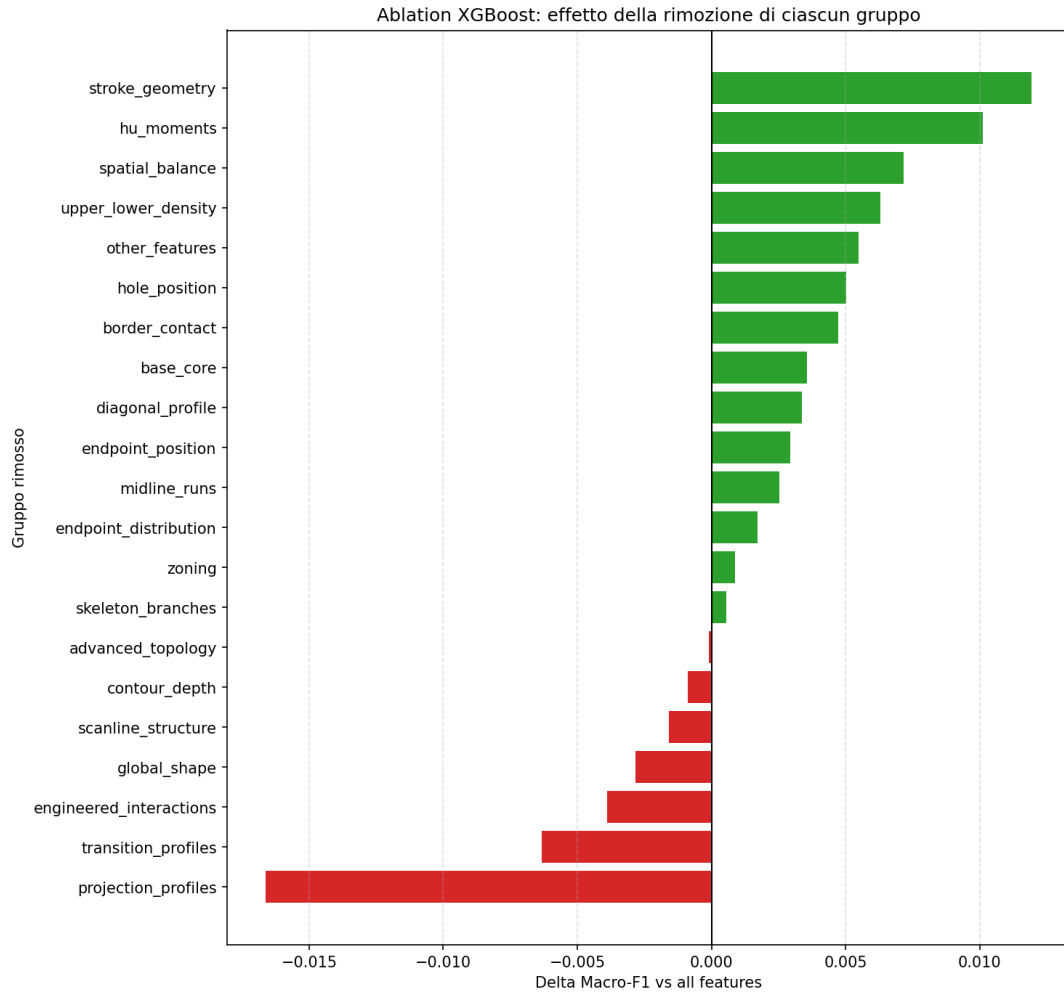


Figura 1: Ablation XGBoost: variazione di Macro-F1 quando si rimuove ciascun gruppo di feature.

Cosa mostra: l'effetto marginale di ogni gruppo sulla qualità del modello. *Cosa ci dice:* le barre negative identificano gruppi informativi, le positive gruppi potenzialmente ridondanti nel setup corrente.

L'immagine in Figura 1 si interpreta così:

- barre a sinistra dello zero (Δ Macro-F1 negativo) indicano gruppi **importanti**: rimuoverli peggiora il modello;
- barre a destra dello zero (Δ Macro-F1 positivo) indicano gruppi potenzialmente **ridondanti** o rumorosi in questo setup.

I risultati più rilevanti sono riportati in Tabella 3.

Tabella 3: Ablation XGBoost: gruppi principali e impatto su Macro-F1.

Gruppo rimosso	Δ Macro-F1 vs all_features
<code>projection_profiles</code>	-0.0166
<code>transition_profiles</code>	-0.0063
<code>engineered_interactions</code>	-0.0039
<code>global_shape</code>	-0.0028
<code>stroke_geometry</code>	+0.0119
<code>hu_moments</code>	+0.0101

Il motivo della perdita è che questi gruppi non descrivono dettagli secondari, ma aspetti strutturali molto informativi della lettera. `projection_profiles` cattura come l'inchiostro si distribuisce lungo righe e colonne, quindi separa bene lettere con massa concentrata in zone diverse; quando lo rimuovo, il modello perde informazione sulla geometria globale. `transition_profiles` misura la complessità locale del contorno e la frequenza di passaggi tra fondo e tratto, perciò aiuta a distinguere lettere con profili simili ma bordi differenti. `engineered_interactions` combina feature base già utili in forma non lineare, quindi la loro rimozione elimina relazioni tra endpoint, buchi e densità che XGBoost sfrutta molto bene. `global_shape` racchiude invece proprietà di forma complessiva come solidità, eccentricità e aspect ratio, cioè segnali importanti per distinguere lettere verticali, arrotondate o allungate.

Al contrario, il miglioramento leggero ottenuto togliendo `stroke_geometry` e `hu_moments` suggerisce che, nello split corrente, queste feature siano in parte ridondanti rispetto ad altri descrittori già presenti oppure più sensibili a variazioni di scrittura e normalizzazione.

In particolare, la rimozione di `projection_profiles` produce la perdita più alta (-0.0166 Macro-F1), quindi questo blocco risulta il più informativo nel setup corrente. Al contrario, la rimozione di `stroke_geometry` e `hu_moments` migliora leggermente la metrica in questo split: questo suggerisce possibile ridondanza e motiva una successiva feature selection mirata.

6 Analisi qualitativa e diagnostica

Per comprendere *come* il modello decide e *dove* sbaglia, l'analisi qualitativa include:

- **heatmap correlazioni feature:** individua ridondanze e gruppi informativi;
- **proiezione PCA 2D:** visualizza la separazione globale tra classi;
- **confusion matrix:** mostra le coppie di lettere critiche;
- **F1 per lettera:** misura quali classi sono deboli/forti in modo puntuale.

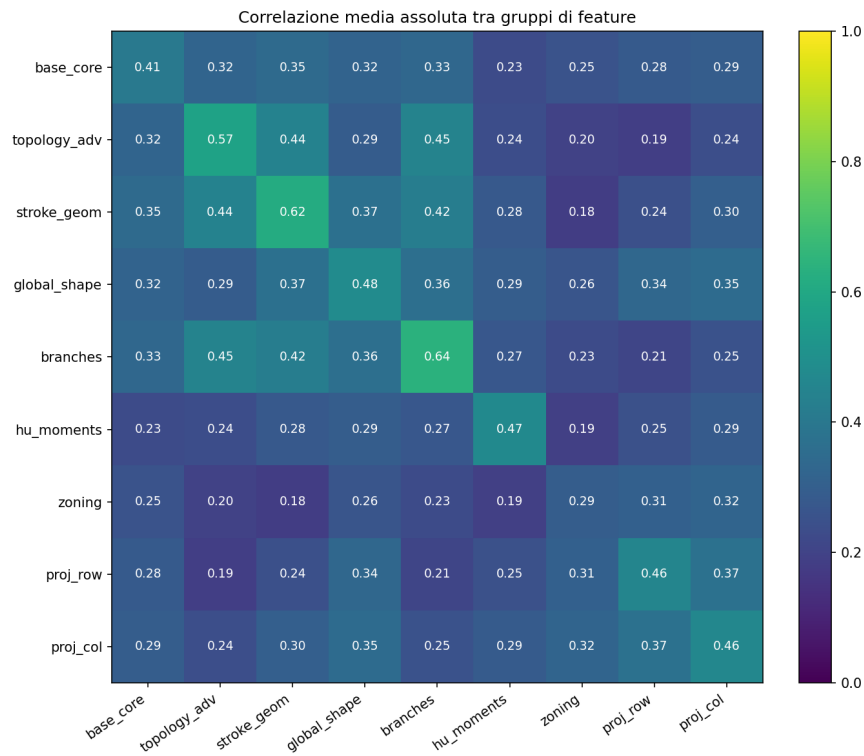


Figura 2: Heatmap di correlazione (gruppi di feature).

Cosa mostra: il livello di correlazione media tra blocchi di feature. *Cosa ci dice:* blocchi molto correlati possono introdurre ridondanza, blocchi meno correlati aggiungono informazione complementare.

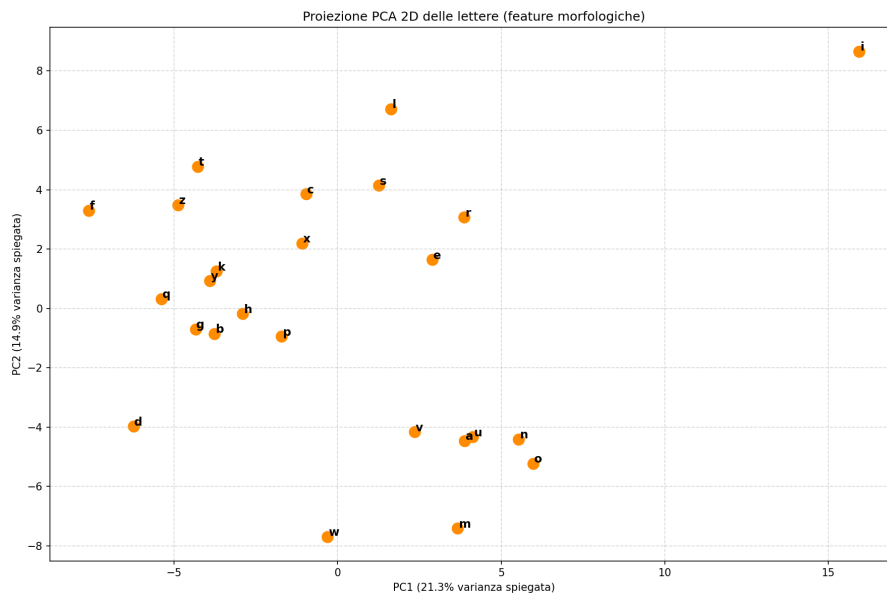


Figura 3: PCA 2D del feature space.

Cosa mostra: la proiezione bidimensionale delle lettere nello spazio delle feature. *Cosa ci dice:* cluster separati indicano buona discriminabilità, sovrapposizioni anticipano possibili confusioni tra classi simili.

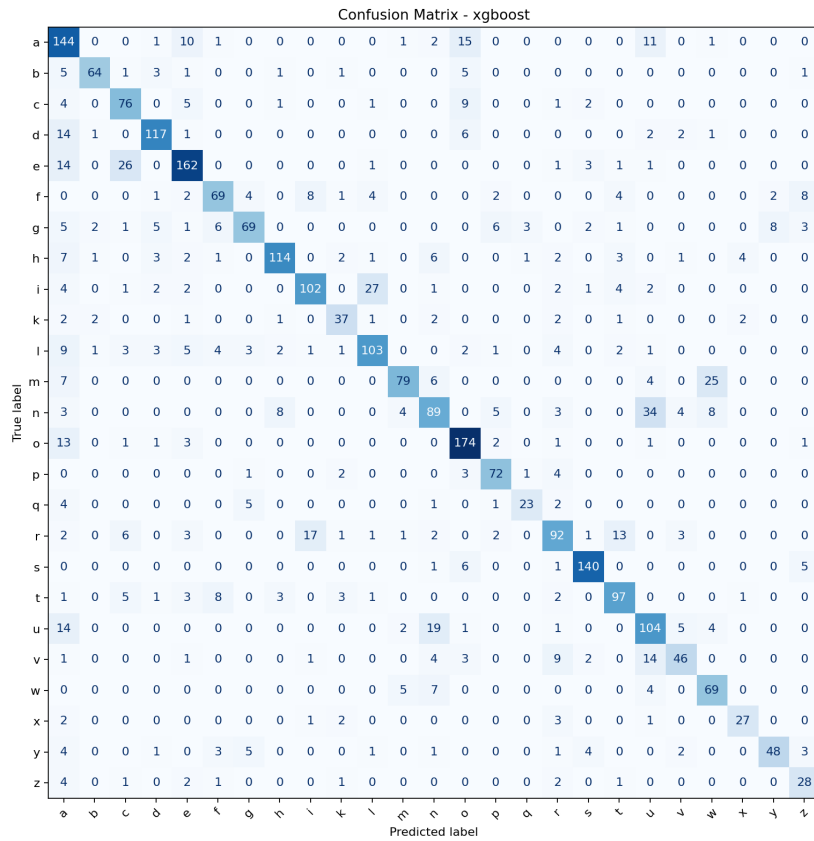


Figura 4: Confusion matrix del modello XGBoost (holdout).

Cosa mostra: conteggi assoluti di predizioni corrette/errate per ogni coppia vera-predetta.

Cosa ci dice: evidenza dove il best model concentra gli errori e quali lettere risultano più stabili.

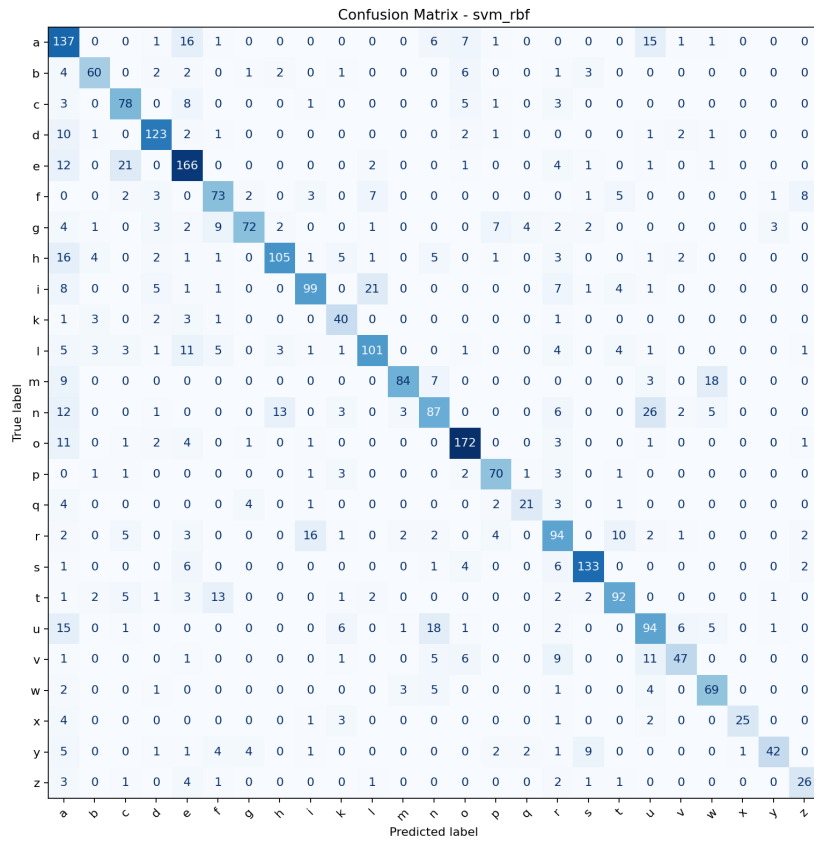


Figura 5: Confusion matrix del modello SVM RBF (holdout).

Cosa mostra: errori e successi assoluti della SVM sulle classi di test. *Cosa ci dice:* permette il confronto diretto con XGBoost sulle stesse coppie di lettere difficili.

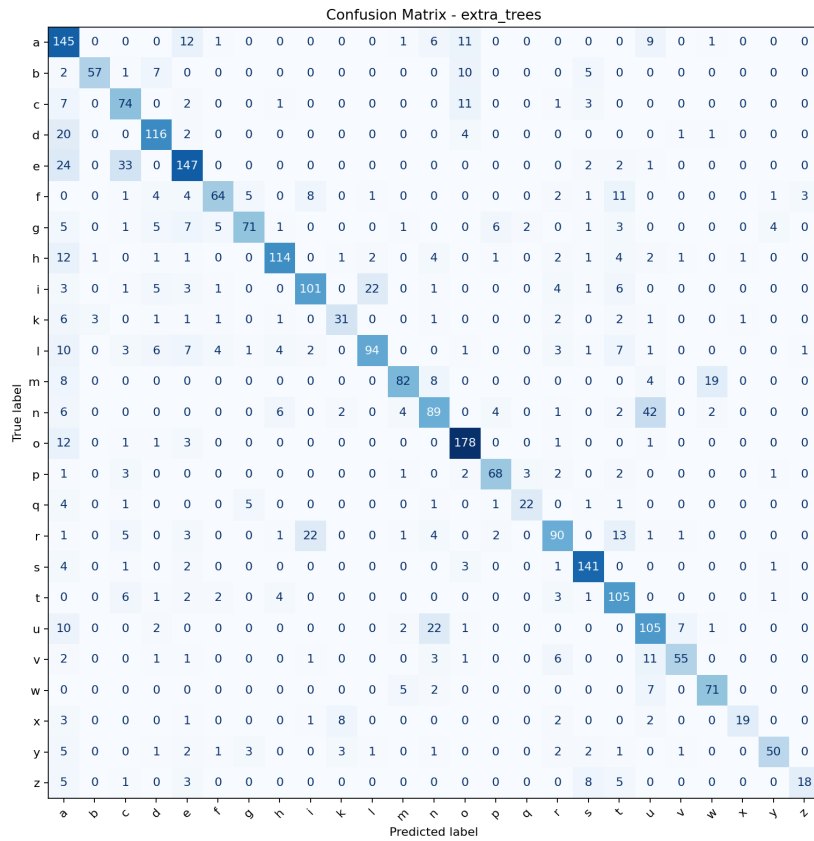


Figura 6: Confusion matrix del modello ExtraTrees (holdout).

Cosa mostra: distribuzione degli errori in valore assoluto per il modello ad alberi randomizzati. *Cosa ci dice:* chiarisce se il comportamento dell'ensemble differisce da SVM/XGBoost su lettere graficamente affini.

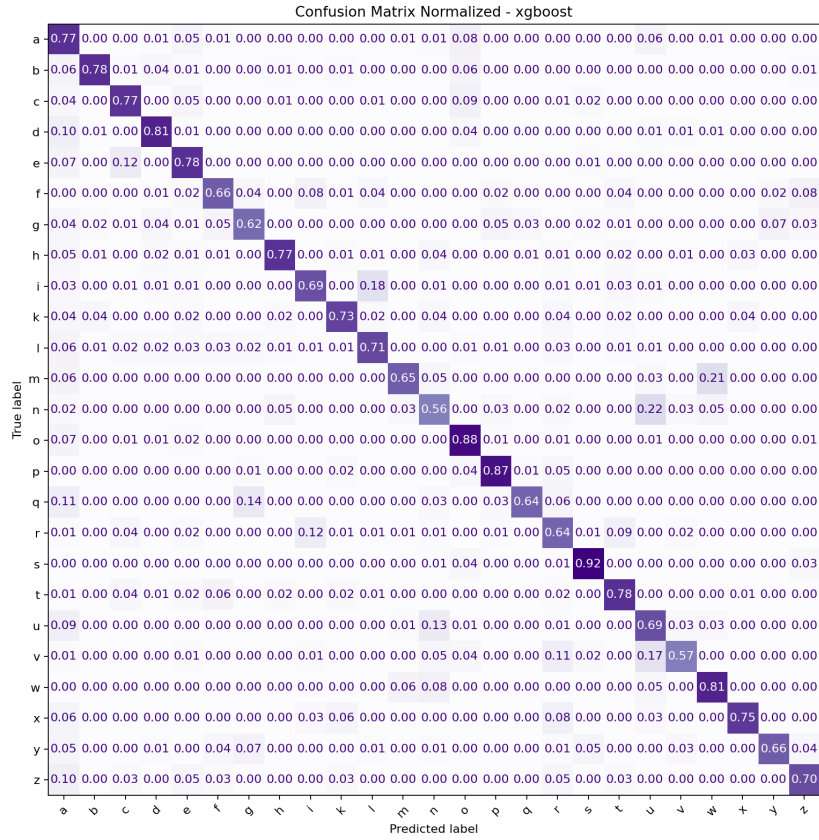


Figura 7: Confusion matrix normalizzata del modello XGBoost.

Cosa mostra: tassi di errore/correttezza per classe indipendenti dalla numerosità. *Cosa ci dice:* consente di vedere quali lettere sono intrinsecamente più complesse da riconoscere anche con supporto diverso.

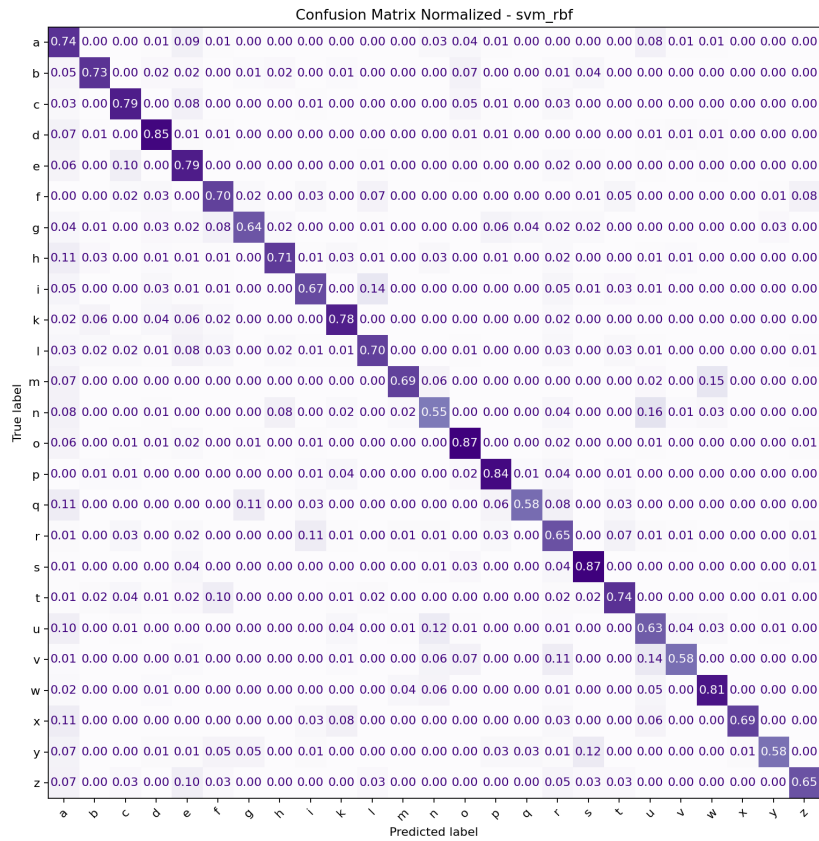


Figura 8: Confusion matrix normalizzata del modello SVM RBF.

Cosa mostra: probabilità di confusione classe-per-classe per la SVM. *Cosa ci dice:* rende confrontabile la robustezza della SVM su classi rare e classi frequenti.

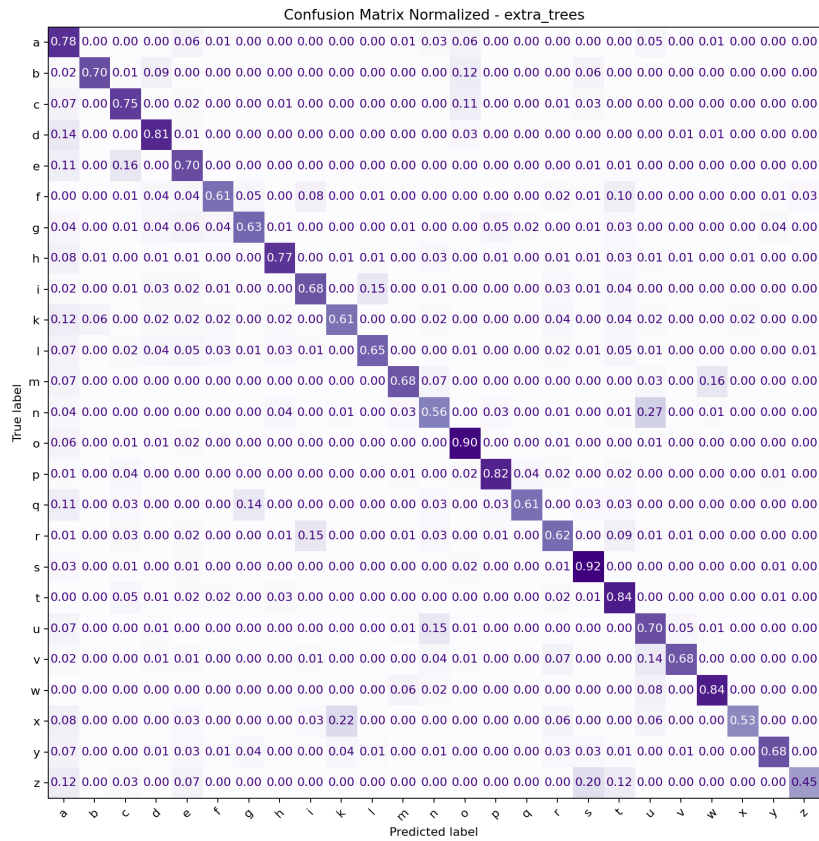


Figura 9: Confusion matrix normalizzata del modello ExtraTrees.

Cosa mostra: profilo di errore relativo del modello ExtraTrees. *Cosa ci dice:* aiuta a identificare quali classi perdono più precisione/recall rispetto agli altri modelli.

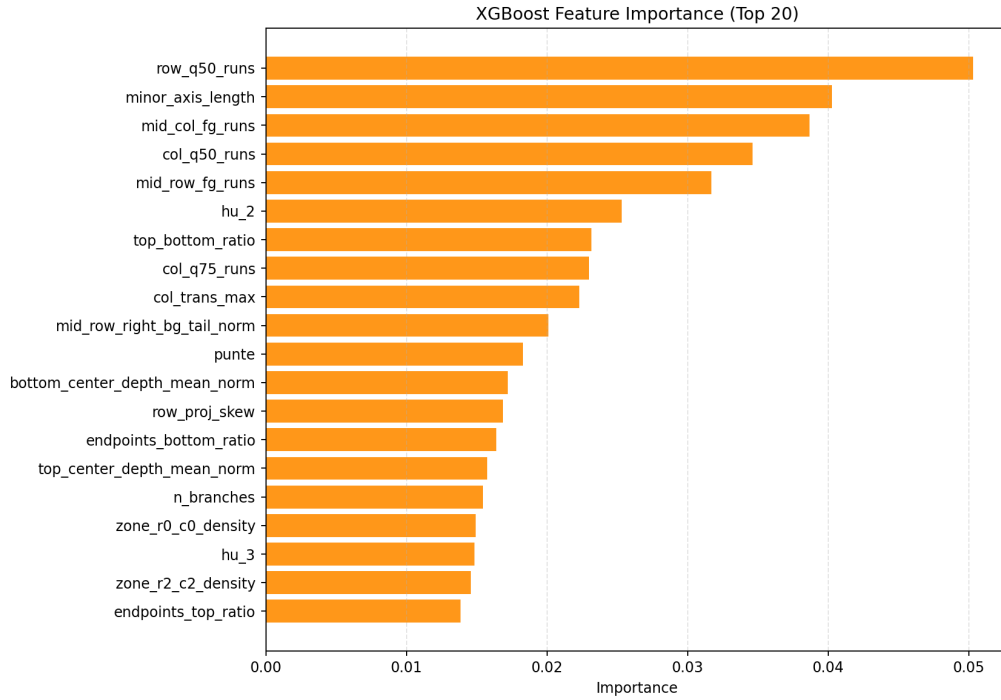


Figura 10: Feature importance (Top-20) del modello XGBoost.

Cosa mostra: le feature con maggior contributo alle decisioni del boosting. *Cosa ci dice:* evidenzia i segnali morfologici più discriminativi e guida eventuale feature selection.

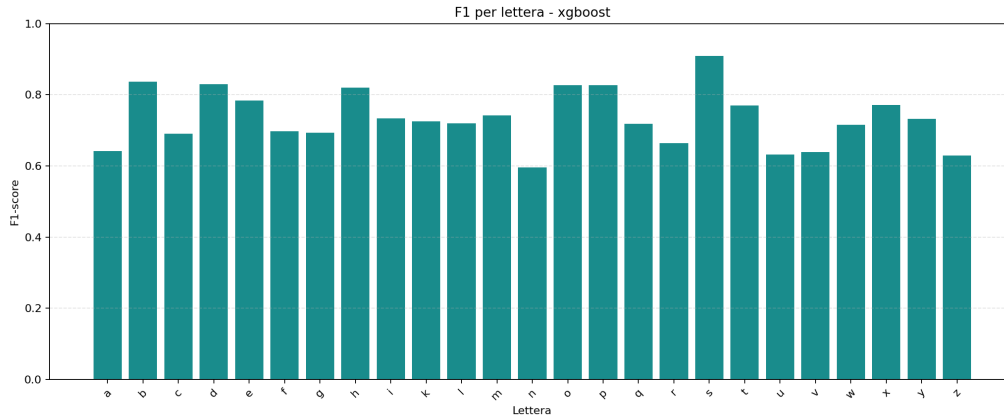


Figura 11: F1-score per lettera del best model.

Cosa mostra: performance della migliore configurazione su ciascuna lettera. *Cosa ci dice:* individua classi forti e classi deboli su cui concentrare miglioramenti mirati.

7 Discussione

I risultati mostrano in modo coerente che:

- il feature engineering morfologico produce prestazioni competitive senza ricorrere a CNN;
- il setup subject-wise riduce in modo sostanziale il rischio di overfitting a uno specifico stile;

- le confusioni residue sono semanticamente plausibili (coppie graficamente simili).

Dal punto di vista metodologico, il valore principale è la tracciabilità della decisione: si può collegare un errore a gruppi specifici di feature e quindi proporre miglioramenti mirati. L’approccio è quindi particolarmente adatto quando si richiedono **interpretabilità**, **diagnostica** e **controllo** del processo.

8 Conclusioni e sviluppi futuri

Il progetto fornisce una pipeline completa, ripetibile e interpretabile per l’analisi e la classificazione di lettere manoscritte. Il contributo non è solo nel valore numerico finale, ma nella costruzione di un processo leggibile in tutte le sue fasi: dalla morfologia all’analisi dell’errore.

Come sviluppi futuri:

1. feature selection mirata sulle classi più deboli (riduzione ridondanza e maggiore robustezza);
2. tuning iperparametri con protocollo nested CV per una stima ancora più rigorosa;
3. estensione ibrida con backbone CNN leggero, mantenendo le feature morfologiche come canale interpretabile;
4. analisi dell’importanza delle feature per gruppo, per guidare nuove iterazioni di design.

A Riproducibilità

A.1 Script principali

- `extract_characters_cv1.py`: segmenta caratteri da immagini CVL (single/batch), applica filtri geometrici/outlier e salva un manifest con bounding box.
- `analyze_letter.py`: esegue preprocessing+morfologia e costruisce il dataset di feature campione per campione.
- `train_noncnn_advanced.py`: allena e confronta SVM/ExtraTrees/XGBoost su split holdout per soggetto.
- `crossval_subjectwise.py`: stima robustezza con CV subject-wise a 5 fold.
- `ablation_noncnn_morphology.py`: studia l’impatto dei gruppi di feature (ablation).

A.2 Output principali

- `output/extracted_characters/extraction_manifest.csv`
- `output/classification_advanced/organized/01_holdout/advanced_summary.csv`
- `output/classification_advanced/organized/01_holdout/advanced_metrics.json`
- `output/classification_advanced/organized/02_cv_subjectwise/cv_subjectwise_summary.csv`

- `output/classification_advanced/organized/03_ablation/ablation_summary.csv`
- `output/classification_advanced/organized/03_ablation/ablation_group_importance_rank`
- `output/classification_advanced/organized/03_ablation/ablation_feature_group_coverag`
- `output/classification_advanced/organized/03_ablation/ablation_delta_macro_f1.png`
- `output/classification_advanced/organized/03_ablation/ablation_report.json`
- `output/classification_advanced/organized/04_diagnostics/confusion_matrices/raw/conf`
- `output/classification_advanced/organized/04_diagnostics/confusion_matrices/raw/conf`
- `output/classification_advanced/organized/04_diagnostics/confusion_matrices/raw/conf`
- `output/classification_advanced/organized/04_diagnostics/confusion_matrices/normaliz`
- `output/classification_advanced/organized/04_diagnostics/confusion_matrices/normaliz`
- `output/classification_advanced/organized/04_diagnostics/confusion_matrices/normaliz`
- `output/classification_advanced/organized/04_diagnostics/feature_importance/xgboost_`

A.3 Procedura minima di esecuzione

1. (Opzionale) Segmentazione caratteri CVL: `python extract_characters_cvl.py -input <path> -output <path> -save-debug -min-area 30 -min-width 5 -min-height 8 -max-aspect-ratio 10 -opening-kernel 2`
2. Generare le feature: `python analyze_letter.py`
3. Allenare e valutare in holdout: `python train_noncnn_advanced.py`
4. Eseguire CV subject-wise: `python crossval_subjectwise.py`
5. (Opzionale) Eseguire ablation: `python ablation_noncnn_morphology.py`